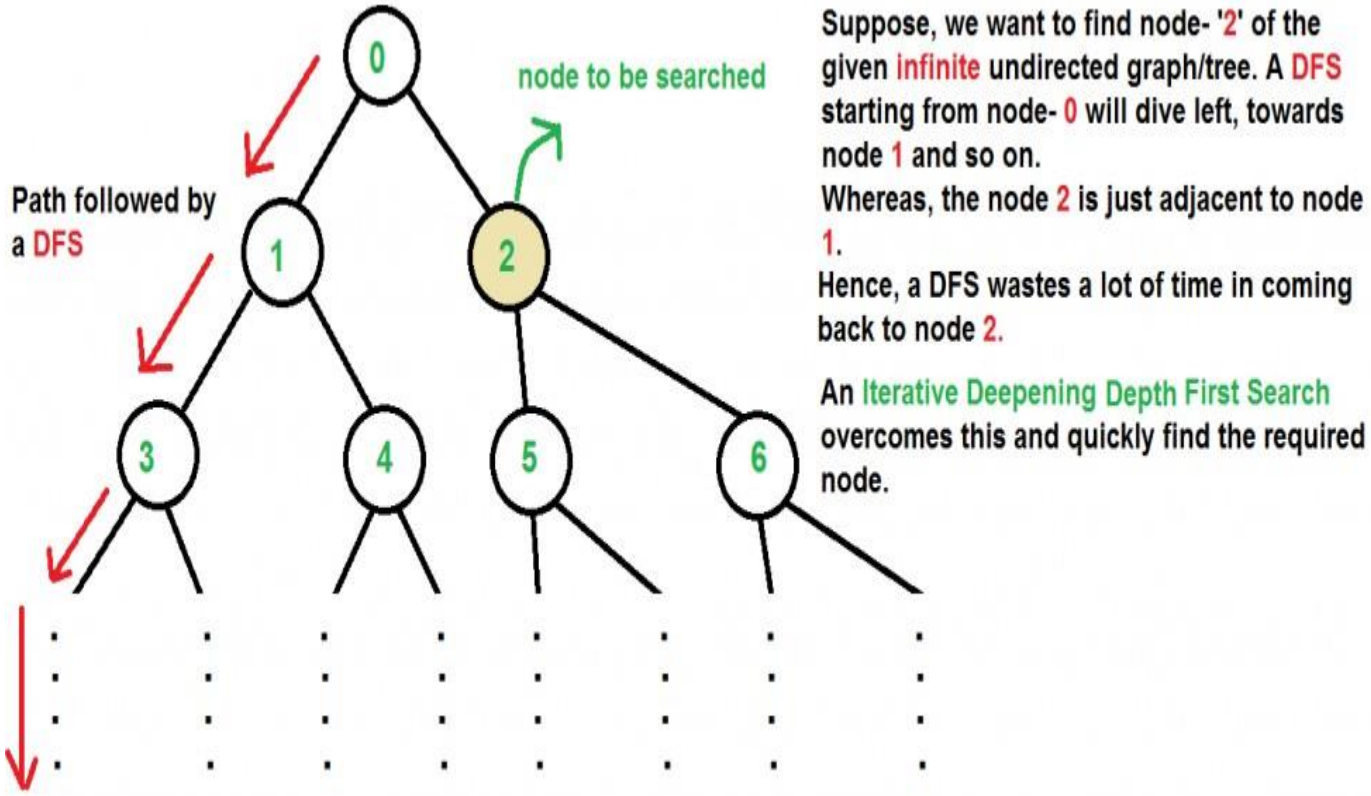
Lab 7

There are two common ways to traverse a graph, BFS and DFS. Considering a Tree (or Graph) of huge height and width, both BFS and DFS are not very efficient due to following reasons.

1. **DFS:** first traverses nodes going through one adjacent of root, then next adjacent. The problem with this approach is, if there is a node close to root, but not in first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find shortest path to a node (in terms of number of edges).



2. **BFS:** goes level by level, but requires more space. The space required by DFS is O(d) where d is depth of tree, but space required by BFS is O(n) where n is number of nodes in tree (Why? Note that the last level of tree can have around n/2 nodes and second last level n/4 nodes and in BFS we need to have every level one by one in queue).

IDDFS combines depth-first search's space-efficiency and breadth-first search's fast search (for nodes closer to root).

2. Objective of the Experiment

After completing this lab, the student should be able to:

- Clearly understand difference between IDS, DLS.
- Better understanding of flaws in BFS & DFS.

3. Concept Map

IDDFS is a combination of both DFS and BFS.

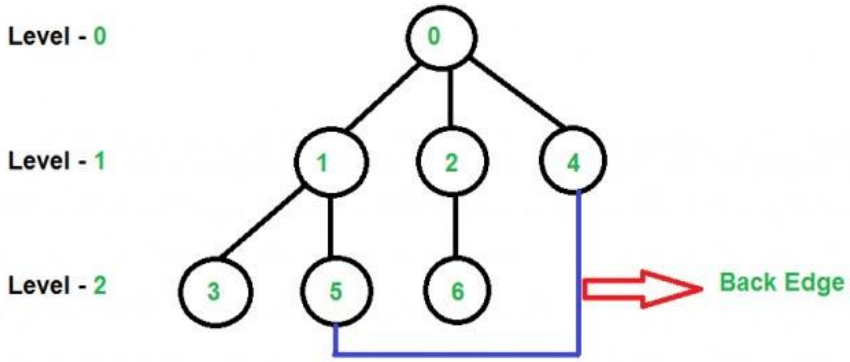
3.1 Depth Limited Search

Depth Limited Search is a basic DFS with a cut-off limit that it stops expanding after.
The problem is how to pick a good limit. Sometimes in DFS due to infinite loop the path to the goal node might never be found, in order to combat this we can add a limit to the depth that our search recurses down the tree, this essentially transforms our depth first algorithm into a depth-limited algorithm

3.2 Iterative Deepening Search/ Iterative Deepening Depth First Search

There can be two cases-
a) When the graph has no cycle: This case is simple. We can DFS multiple times with different height limits.

b) When the graph has cycles: This is interesting as there is no visited flag in IDDFS.



Although, at first sight, it may seem that since there are only 3 levels, so we might think that Iterative Deepening Depth First Search of level 3, 4, 5,...and so on will remain same. But, this is not the case. You can see that there is a cycle in the above graph, hence IDDFS will change for level-3,4,5..and so on.

Figure 1: IDDFS

Depth	Iterative Deepening Depth First Search
0	0
1	0 1 2 4
2	0 1 3 5 2 6 4 5
3	0 1 3 5 4 2 6 4 5 1

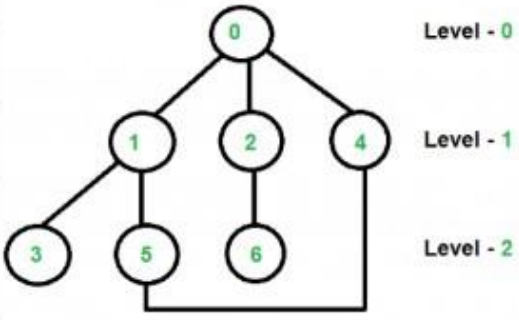


Figure 2: IDDFS search path

A comparison table between DFS, BFS and IDDFS

	Time Complexity	Space Complexity	When to Use ?
DFS	O(b^d)	O (d)	=> Don't care if the answer is closest to the starting vertex/root. => When graph/tree is not very big/infinite.
BFS	O(b^d)	O(b^d)	=> When space is not an issue => When we do care/want the closest answer to the root.
IDDFS	O(b^d)	O(bd)	=> You want a BFS, you don't have enough memory, and somewhat slower performance is accepted. In short, you want a BFS + DFS.

Figure 3: DFS, BFS and IDDFS Comparision

4. Procedure & Tools

4.1 Tools

Anaconda

4.2 Iterative Deepening Search Algorithm

IDDFS calls DFS for different depths starting from an initial value. In every call, DFS is restricted from going beyond given depth. So basically we do DFS in a BFS fashion.

```
// Returns true if target is reachable from
// src within max_depth
bool IDDFS(src, target, max_depth)
    for limit from 0 to max_depth
       if DLS(src, target, limit) == true
           return true
    return false

bool DLS(src, target, limit)
    if (src == target)
        return true;

    // If reached the maximum depth,
    // stop recursing.
```

```
if (limit <= 0)
    return false;

foreach adjacent i of src
    if DLS(i, target, limit?1)
        return true

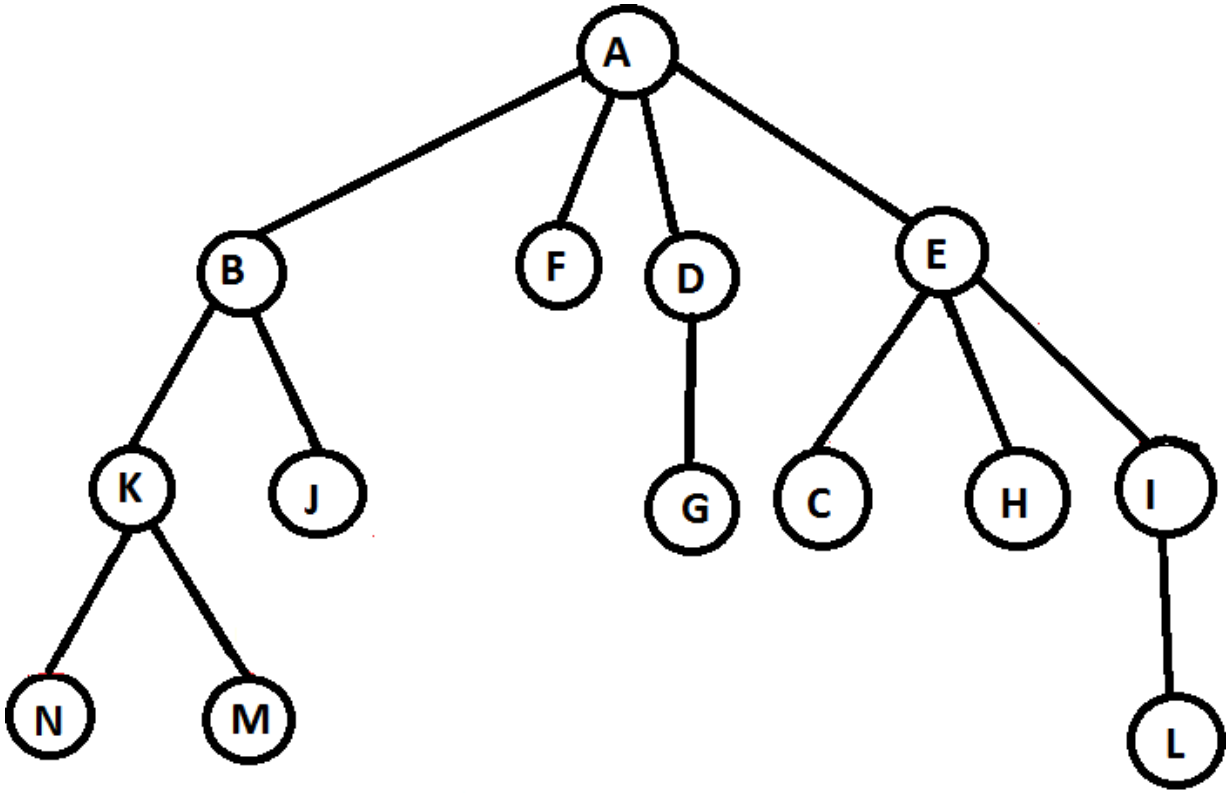
return false
```

An important thing to note is, we visit top level nodes multiple times. The last (or max depth) level is visited once, second last level is visited twice, and so on. It may seem expensive, but it turns out to be not so costly, since in a tree most of the nodes are in the bottom level. So it does not matter much if the upper levels are visited multiple times

Tasks

4.3 Task 1

You are given the following tree whose starting node is **A** and Goal node is **G.**



You are required to apply the following searches and submit in MS Word file.

a) Depth Limited Search
b) Iterative Deepening Search.
c) Compare searches and conclude which search gave the best path in the above situation.